

词典

桶排序： 算法

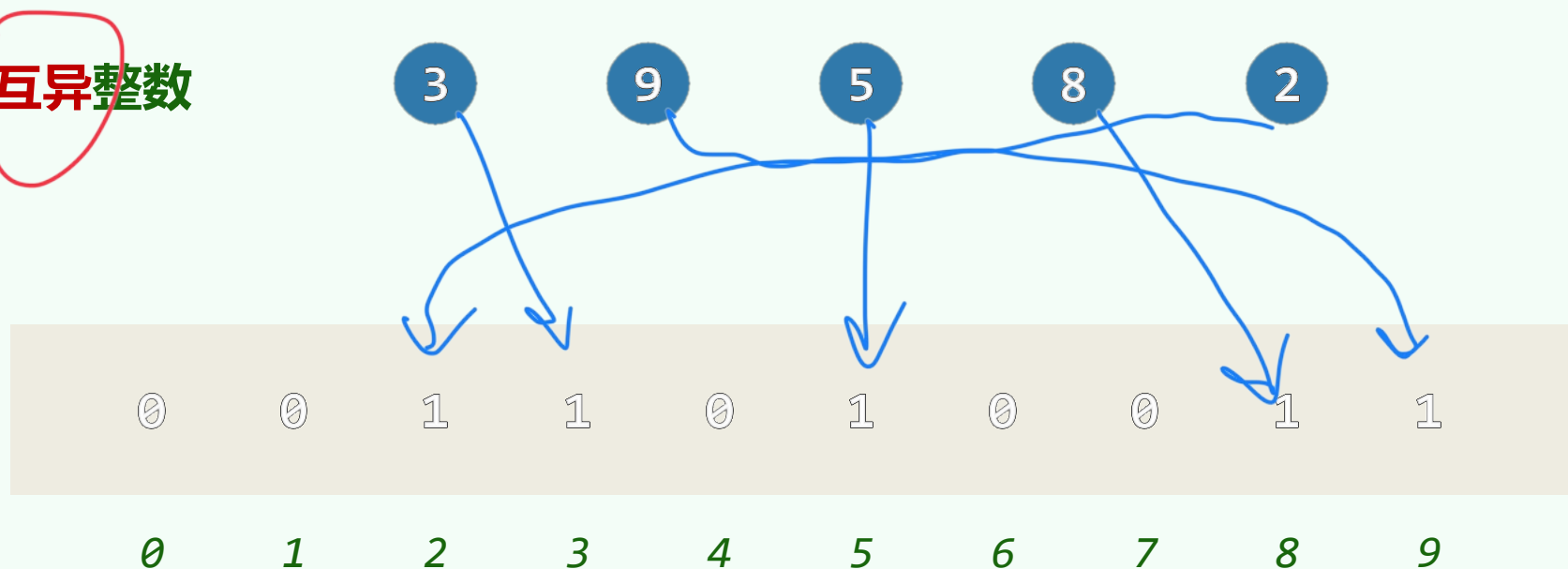
简单情况

❖ 对 $[0, m)$ 内的 n ($< m$) 个互异整数

借助散列表 $\mathcal{H}[]$ 做排序

❖ 空间 = $O(m)$

时间 = $O(n)$!



❖ initialization: for $i = 0$ to $m - 1$, let $\mathcal{H}[i] = 0$ // $O(m)$ --> $O(1)$

distribution: for each key in the input, let $\mathcal{H}[\text{key}] = 1$ // $O(n)$

有多少个数要放多少次

enumeration: for $i = 0$ to $m - 1$, output i if $\mathcal{H}[i] = 1$ // $O(m)$ --> $O(n)$

对每个桶都要查一次

一般情况

❖ 若允许重复

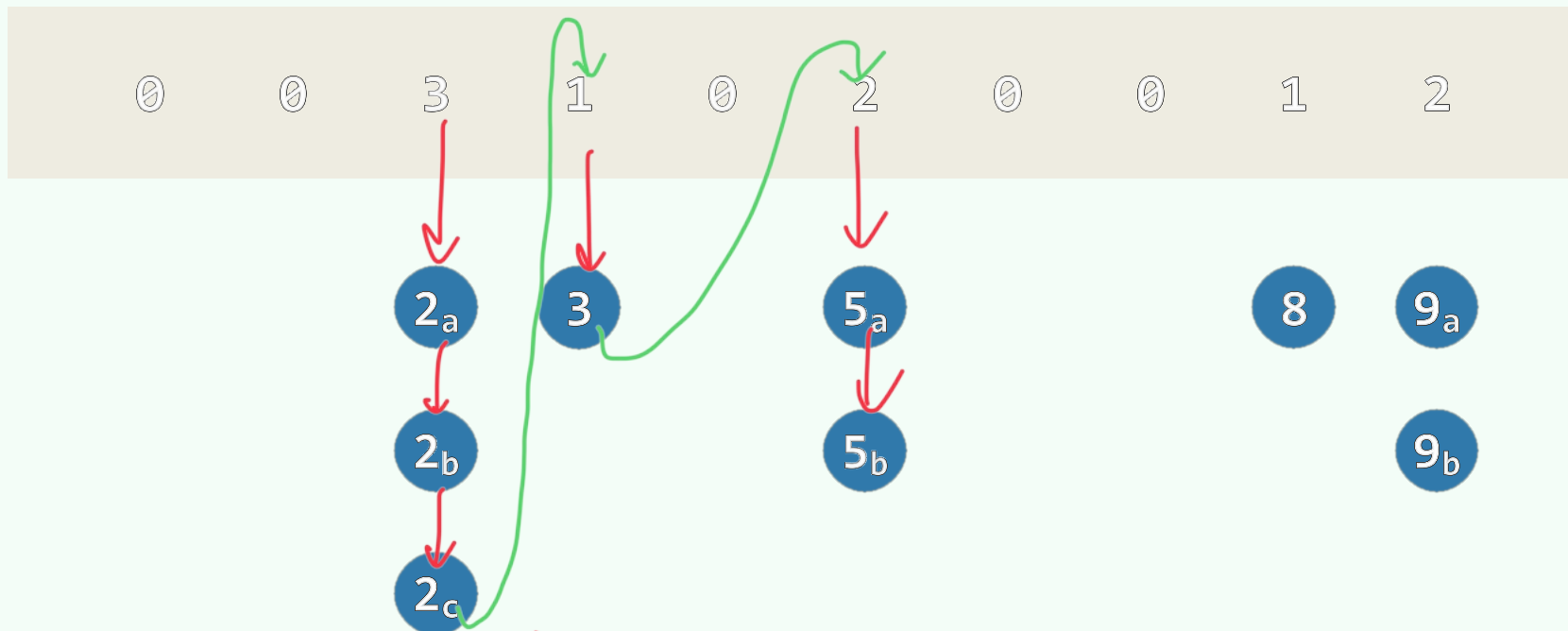
(可能 $m \ll n$)

❖ 依然使用散列表

- 每一组同义词
- 各成一个链表

❖ 空间 = $O(m + n)$

时间 = $O(n)$!



按先后到的顺序串起来(排序稳定性)

词典

桶排序： 最大缝隙

MaxGap

❖ 任意 n 个互异点均将实轴分为 $n-1$ 段有界区间，其中的哪一段**最长**？

❖ 如果不追求效率，显而易见的方法莫过于...



- 对所有点**排序** //可惜，这需要 $\Omega(n \log n)$ 时间
- 依次计算各相邻点对的间距，保留最大者 // $\Theta(n)$

❖ 可否更快？

❖ 采用分桶策略，可改进至 $\mathcal{O}(n)$ 时间...

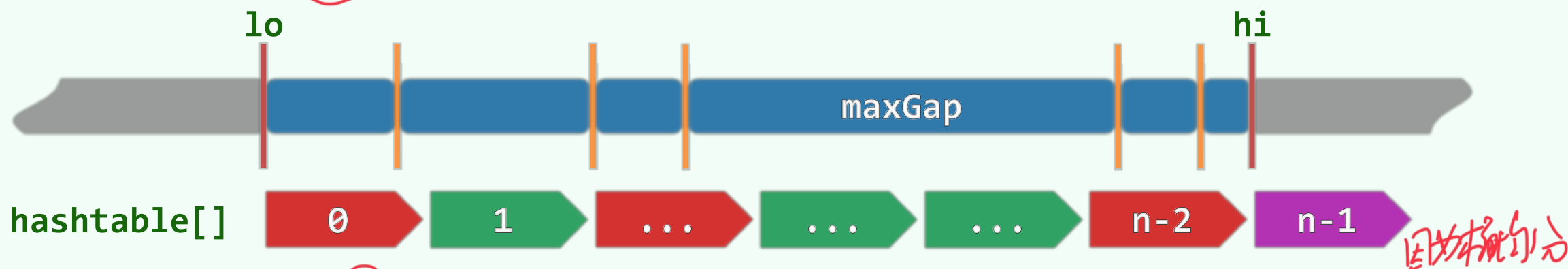
线性算法

找到最左点、最右点

$O(n)$ //一趟线性扫描

将有效范围均匀地划分为 $n-1$ 段 (n 个桶)

$O(n)$ //相当于散列表



通过散列，将各点归入对应的桶

$O(n)$ //模余法

不用第一个桶内的Gap

在各桶中，动态记录最左点、最右点

$O(n)$ //可能相同甚至没有

算桶桶之间即可

算出相邻 (非空) 桶之间的“距离”

$O(n)$ //一趟遍历足矣

例如: 左桶右边界 ~ 右桶左边界

最大的距离即MaxGap

$O(n)$ //画家算法

词典

基数排序： 算法与实现

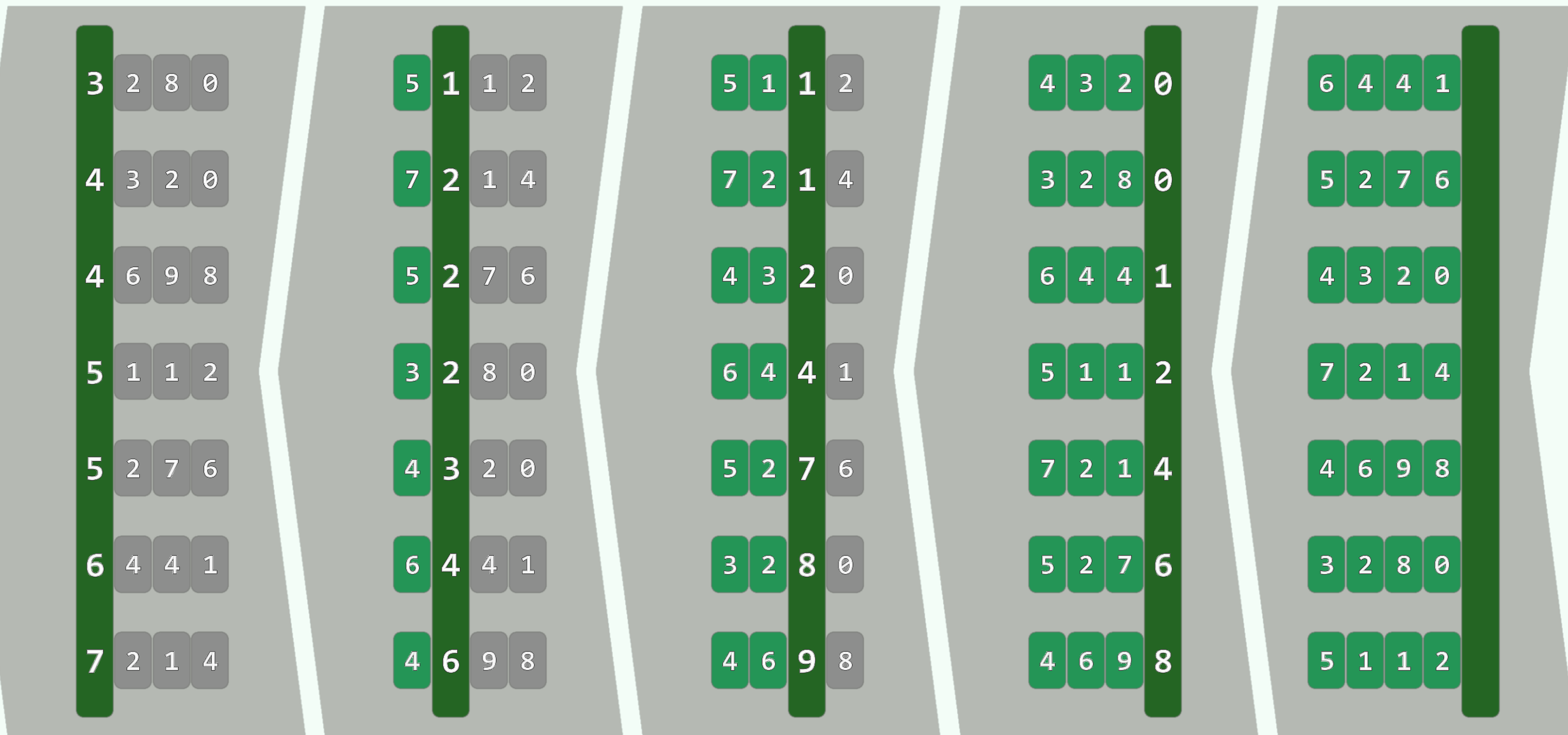
词典序

❖ 有时，关键码由多个域组成： k_d , k_{t-1} , ... , k_1

❖ 若将各域视作字母，则关键码即单词——按词典的方式排序 (lexicographic order)



算法：自 k_1 到 k_t （**低位优先**），依次以各域为序做一趟**桶排序**



正确性

3	2	8	0
4	3	2	0
4	6	9	8
5	1	1	2
5	2	7	6
6	4	4	1
7	2	1	4

5	1	1	2
7	2	1	4
5	2	7	6
3	2	8	0
4	3	2	0
6	4	4	1
4	6	9	8

❖ 归纳假设：前 i 趟排序后，所有词条关于低 i 位有序
(第1趟显然)

❖ 假设前 $i-1$ 趟均成立，现考查第 i 趟排序之后的时刻

❖ 无非两种情况

- 凡第 i 位不同的词条

即便此前曾是逆序，现在亦必已转为有序

- 凡第 i 位相同的词条

得益于桶排序的稳定性，必保持原有次序

时间成本

3	2	8	0
4	3	2	0
4	6	9	8
5	1	1	2
5	2	7	6
6	4	4	1
7	2	1	4

5	1	1	2
7	2	1	4
5	2	7	6
3	2	8	0
4	3	2	0
6	4	4	1
4	6	9	8

= 各趟桶排序所需时间之和

$$= n + 2m_1$$

$$+ n + 2m_2$$

$$+ \dots$$

$$+ n + 2m_d \quad // m_k \text{ 为各域的取值范围}$$

$$= O(d \times (n + m))$$

$$// m = \max\{ m_1, \dots, m_d \}$$

❖ 当 $m = O(n)$ 且 d 可视作常数时, $O(n)$!

❖ 在一些特定场合, Radixsort 非常高效 // 稍后举例

分散: n : 与排序个数有关

收集: m_i : 与每次桶个数有关

$n \gg m$

实现（以二进制无符号整数为例）

❖ `typedef unsigned int U; //约定：类型T或就是U；或可转换为U，并依此定序`

❖ `template <typename T> void List<T>::radixSort( ListNodePosi<T> p, int n ) {`

`ListNodePosi<T> head = p->pred; ListNodePosi<T> tail = p;`

`for ( int i = 0; i < n; i++ ) tail = tail->succ; //待排序区间为(head, tail)`

`for ( U radixBit = 0x1; radixBit && (p = head); radixBit <<= 1 ) //以下反复地`

`for ( int i = 0; i < n; i++ ) //根据当前基数位，将所有节点`

`radixBit & U (p->succ->data) ? //分拆为前缀 (0) 与后缀 (1)`

`insert( remove( p->succ ), tail ) : p = p->succ;`

`} //为避免remove()、insert()的低效率，可拓展List::move(p,tail)接口，将节点p直接移至tail之前`

实例

H	1 _a	0 _a	1 _b	1 _c	0 _b	0 _c	0 _d	1 _d	0 _e	0 _f	1 _e	1 _f	0 _g	T
---	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	---

只有两个桶

H		0 _a			0 _b	0 _c	0 _d		0 _e	0 _f			0 _g	
---	--	----------------	--	--	----------------	----------------	----------------	--	----------------	----------------	--	--	----------------	--

	1 _a		1 _b	1 _c				1 _d			1 _e	1 _f		T
--	----------------	--	----------------	----------------	--	--	--	----------------	--	--	----------------	----------------	--	---

H	0 _a	0 _b	0 _c	0 _d	0 _e	0 _f	0 _g	1 _a	1 _b	1 _c	1 _d	1 _e	1 _f	T
---	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	----------------	---